

UCS 503 – Software Engineering Project Report

ICU Patient Face Monitoring System

**A Computer-Vision-Based Non-Contact System for Continuous Facial
and Ocular Monitoring of Intensive Care Unit Patients**

Submitted by:

1024240039 Harjot Singh

1024240103 Pahul Singh

BE Third Year, Data Science and Artificial Intelligence

Submitted to:

Ms. Anushka

Project Report

**Computer Science and Engineering Department
Thapar Institute of Engineering and Technology, Patiala**

Abstract

This report presents the design and implementation of the Face + Eye component of an ICU Patient Monitoring System. The objective of the component is to provide a non-contact computer-vision pipeline capable of transforming camera observations of a patient into structured and time-aware facial and ocular monitoring information.

The implemented pipeline begins with camera-frame acquisition and progresses through face detection, facial landmark extraction, eye landmark extraction, Eye Aspect Ratio (EAR) calculation, eye-state classification, blink analysis, temporal processing, gaze estimation, confidence estimation, global-value generation, alert generation, and backend integration.

A key design principle is that individual video frames should not be treated as independent monitoring observations. Eye measurements can be affected by landmark noise, illumination, head movement, temporary occlusion, and camera positioning. The system therefore uses temporal history and duration-based reasoning to distinguish short eye closures from persistent closure conditions.

The communication layer is implemented using a FastAPI backend. The Face + Eye processing pipeline sends structured feature information through an HTTP/JSON interface, while dashboard frames are transmitted through a separate background sender. This separation reduces the effect of dashboard streaming delays on the main processing pipeline.

The current project scope covers the Face + Eye processing pipeline and its integration with the backend API. Full clinical validation, patient-specific calibration, large-scale deployment, historical storage, and production dashboard integration remain future work.

Keywords: ICU monitoring, computer vision, facial landmarks, Eye Aspect Ratio, blink detection, gaze estimation, temporal processing, FastAPI, OpenCV, Python.

List of Abbreviations

Abbreviation	Meaning
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
EAR	Eye Aspect Ratio
FPS	Frames Per Second
HTTP	Hypertext Transfer Protocol
ICU	Intensive Care Unit
JSON	JavaScript Object Notation
REST	Representational State Transfer
TP	True Positive
FP	False Positive
FN	False Negative

Contents

Abstract	i
List of Abbreviations	ii
1 Introduction	1
1.1 Background and Context	1
1.2 Problem Statement	2
1.3 Motivation	2
1.4 Project Objectives	3
1.5 Scope and Boundaries	3
1.6 Project Contributions	4
2 System Analysis and Requirements	6
2.1 System Context	6
2.2 Functional Requirements	7
2.3 Non-Functional Requirements	8
2.4 System Constraints	8
3 Methodology and System Design	9
3.1 Overall Architecture	9
3.2 Context-Level Data Flow	10
3.3 Level-1 Data Flow Diagram	11
3.4 Use Case Design	12
3.5 UML Class Design	13
3.6 Activity Flow	14
3.7 Sequence Flow	15
3.8 Technical Stack	16
4 Implementation	17
4.1 Project Structure	17
4.2 Face Detection	18

4.3	Facial Landmark Detection	18
4.4	Eye Landmark Extraction	19
4.5	Eye Aspect Ratio	19
4.6	Eye-State Classification	19
4.7	Blink Detection	20
4.8	Temporal Processing	20
4.9	Gaze Estimation	21
4.10	Confidence Estimation	21
4.11	Global Monitoring State	21
4.12	Alert Generation	22
5	Backend Integration	23
5.1	Integration Objective	23
5.2	FastAPI Backend	23
5.3	Feature Communication	23
5.4	Dashboard Frame Communication	24
5.5	Asynchronous Frame Delivery	24
5.6	Backend API	24
5.7	End-to-End Integration Flow	26
5.8	Failure Handling	27
6	Testing and Evaluation	28
6.1	Testing Methodology	28
6.2	Performance Metrics	28
6.3	Evaluation Targets	29
6.4	Backend Integration Validation	29
6.5	Current Results	30
7	Discussion	31
7.1	Engineering Significance	31
7.2	Importance of Temporal Reasoning	31
7.3	Processing and Communication Separation	31
7.4	Asynchronous Frame Streaming	31
7.5	Limitations	32

8 Conclusion and Future Work	33
8.1 Summary	33
8.2 Key Findings	33
8.3 Future Work	33
8.4 Final Remarks	34
Bibliography	35

List of Figures

3.1	Context-level Data Flow Diagram of the ICU Face Monitoring System.	10
3.2	Level-1 Data Flow Diagram of the Face + Eye processing pipeline.	11
3.3	Use Case Diagram of the ICU Face Monitoring System.	12
3.4	UML Class Diagram of the Face + Eye processing pipeline.	13
3.5	Activity diagram for per-frame processing and prolonged eye-closure alert generation.	14
3.6	Sequence diagram showing the journey of a frame through the processing pipeline.	15

List of Tables

2.1	Functional requirements	7
2.2	Non-functional requirements	8
3.1	Technology stack	16
4.1	EAR-based eye-state interpretation	20
4.2	Observation-level alert categories	22
6.1	Testing methodology	28
6.2	Proposed evaluation targets	29

Introduction

1.1 Background and Context

Intensive Care Units (ICUs) require continuous observation because the condition of a patient can change rapidly. Conventional ICU monitoring systems primarily rely on physiological sensors such as ECG electrodes, pulse oximeters, blood-pressure monitors, and other medical devices. These systems provide important physiological measurements but do not directly capture visible facial and ocular behaviour.

Computer vision provides an opportunity to supplement conventional monitoring through non-contact observation. A camera can continuously capture the patient's face, while computer-vision algorithms can identify facial landmarks and extract information from the eye region. Vision-based patient monitoring has previously been studied as a possible supplement to conventional monitoring systems [BDMS16; DMS+19].

The Face + Eye component developed as part of the ICU Patient Monitoring System follows this approach. Rather than stopping at face detection, the system progressively converts raw video frames into structured monitoring information.

**Camera → Face Detection → Landmarks → Eye Features → Temporal
Processing → Global Values → Alerts → Backend**

The system separates visual processing from communication. The computer-vision layer focuses on extracting observations, while the backend provides an interface through which the processed information can be consumed by downstream components.

The system is intended as a supplementary monitoring and decision-support system. It is not intended to replace professional medical judgement or independently diagnose a patient's medical condition.

1.2 Problem Statement

Continuous visual observation of ICU patients can be challenging when healthcare professionals must monitor several patients simultaneously. Facial and ocular changes may occur continuously and may not always be noticed immediately through manual observation.

The project therefore addresses the following problem:

How can facial and ocular information obtained from a camera be automatically transformed into structured, time-aware monitoring information and made available through a backend service for continuous ICU observation?

The problem consists of several engineering tasks:

1. detecting the patient's face;
2. extracting facial landmarks;
3. extracting eye-region landmarks;
4. calculating eye-related geometric features;
5. classifying observable eye states;
6. analysing eye behaviour over time;
7. estimating relative gaze behaviour;
8. generating global monitoring values;
9. generating observation-level alerts; and
10. communicating processed information to a backend.

1.3 Motivation

A single video frame is often insufficient for reliable behavioural interpretation. A low EAR value can result from an actual blink, incorrect landmarks, temporary occlusion, head movement, or poor image quality.

Consequently, temporal processing is required to determine whether a condition persists over multiple frames.

A second motivation is software modularity. The monitoring interface should not need to know how facial landmarks were detected or how EAR was calculated. A backend API establishes a clear abstraction boundary between the computer-vision engine and downstream applications.

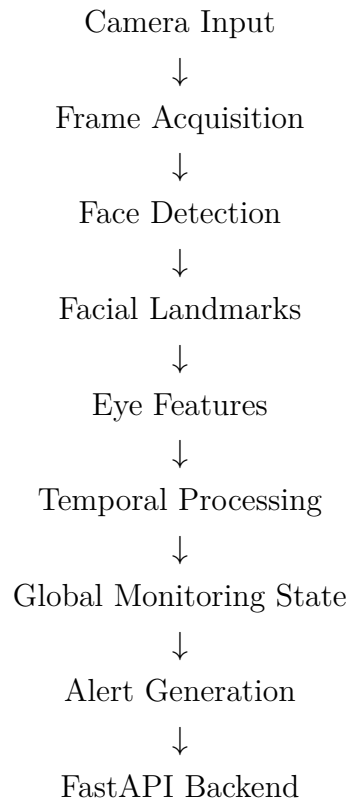
1.4 Project Objectives

The Face + Eye implementation is organised around the following objectives:

1. **Face Detection:** Identify the patient's face from incoming video frames.
2. **Facial Landmark Detection:** Obtain structured facial landmark coordinates.
3. **Eye Landmark Extraction:** Extract landmarks associated with the left and right eyes.
4. **Eye Feature Extraction:** Calculate EAR and related geometric measurements.
5. **Eye-State Analysis:** Distinguish observable states such as open, partially closed, and closed.
6. **Temporal Analysis:** Analyse observations across multiple frames.
7. **Global Value and Alert Generation:** Convert low-level measurements into compact monitoring values and observation-level alerts.
8. **Backend Integration:** Expose processed information through a FastAPI-based communication layer.

1.5 Scope and Boundaries

The current implementation covers the Face + Eye processing chain up to the backend API boundary.



The system does not claim independent medical diagnosis or replacement of professional clinical judgement.

1.6 Project Contributions

The main contributions of the implementation are:

- modular decomposition of the computer-vision pipeline;
- facial and ocular feature extraction;
- EAR-based eye-state analysis;
- temporal reasoning for blink and closure behaviour;
- relative gaze estimation;
- confidence and global-state generation;
- observation-level alert generation;

- separation of processing and communication layers;
- HTTP/JSON backend integration; and
- asynchronous dashboard-frame transmission.

System Analysis and Requirements

2.1 System Context

The Face + Eye component forms one processing component of the larger ICU monitoring architecture. The camera provides visual input, the computer-vision pipeline converts the input into structured observations, and the backend provides these observations to downstream applications.

The principal entities are:

- **Camera:** Provides patient video frames.
- **Face + Eye Engine:** Performs computer-vision processing.
- **Backend:** Receives and exposes processed information.
- **Healthcare Staff:** Consume monitoring information and alerts.
- **Administrator:** Configures system parameters where applicable.

2.2 Functional Requirements

Table 2.1: Functional requirements

ID	Requirement
FR1	The system shall acquire video frames from a camera or video source.
FR2	The system shall detect the patient's face.
FR3	The system shall obtain facial landmarks.
FR4	The system shall extract landmarks associated with both eyes.
FR5	The system shall calculate eye-related geometric features.
FR6	The system shall classify observable eye states.
FR7	The system shall analyse eye behaviour temporally.
FR8	The system shall detect prolonged eye-closure conditions.
FR9	The system shall generate structured global monitoring values.
FR10	The system shall transmit processed values to the backend.
FR11	The system shall support dashboard-frame transmission.

2.3 Non-Functional Requirements

Table 2.2: Non-functional requirements

Property	Requirement
Performance	The system should operate close to real-time.
Modularity	Processing stages should remain independently maintainable.
Reliability	Temporary communication failures should not terminate the main processing pipeline.
Maintainability	Components should have clearly separated responsibilities.
Scalability	The architecture should permit future multi-patient deployment.
Security	Patient information should be transmitted and stored using appropriate controls.
Interpretability	Outputs should represent observable computer-vision states rather than unsupported clinical diagnoses.

2.4 System Constraints

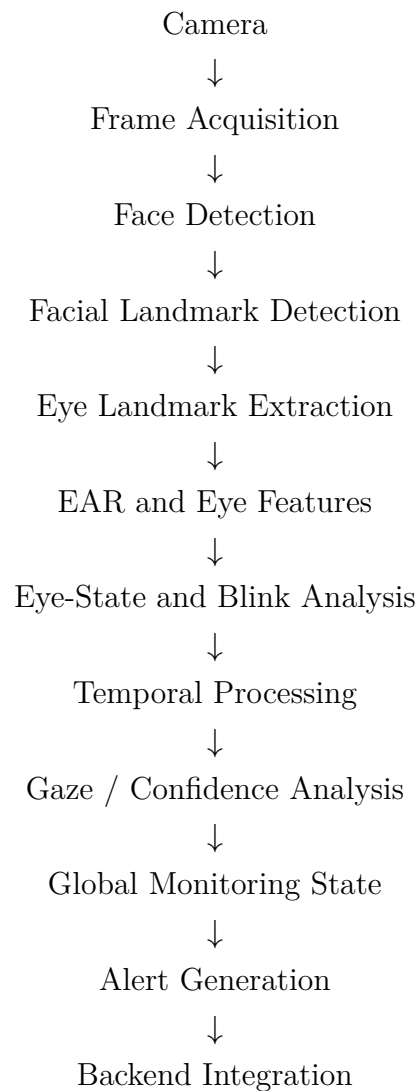
The system can be affected by:

- camera placement;
- illumination;
- patient head orientation;
- facial occlusion;
- landmark quality;
- camera-to-patient distance;
- computational resources; and
- backend/network availability.

Methodology and System Design

3.1 Overall Architecture

The system follows a modular processing architecture. Each stage has a specific responsibility and passes structured information to the next stage.



3.2 Context-Level Data Flow

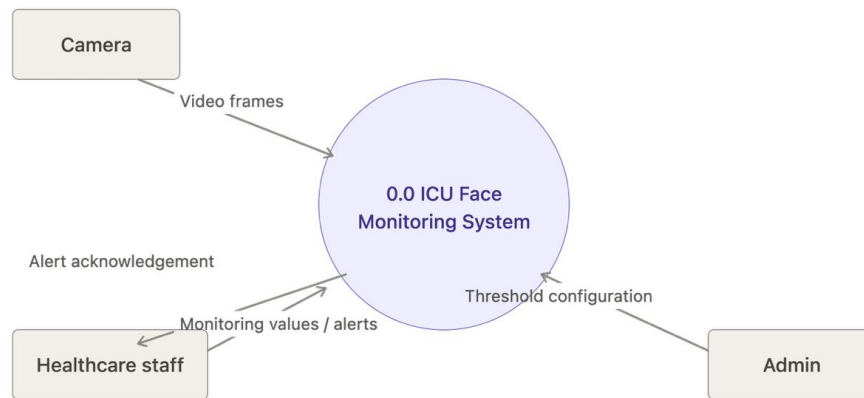


Figure 3.1: Context-level Data Flow Diagram of the ICU Face Monitoring System.

The context-level DFD represents the monitoring system as a single process interacting with the camera, healthcare staff, and administrator.

3.3 Level-1 Data Flow Diagram

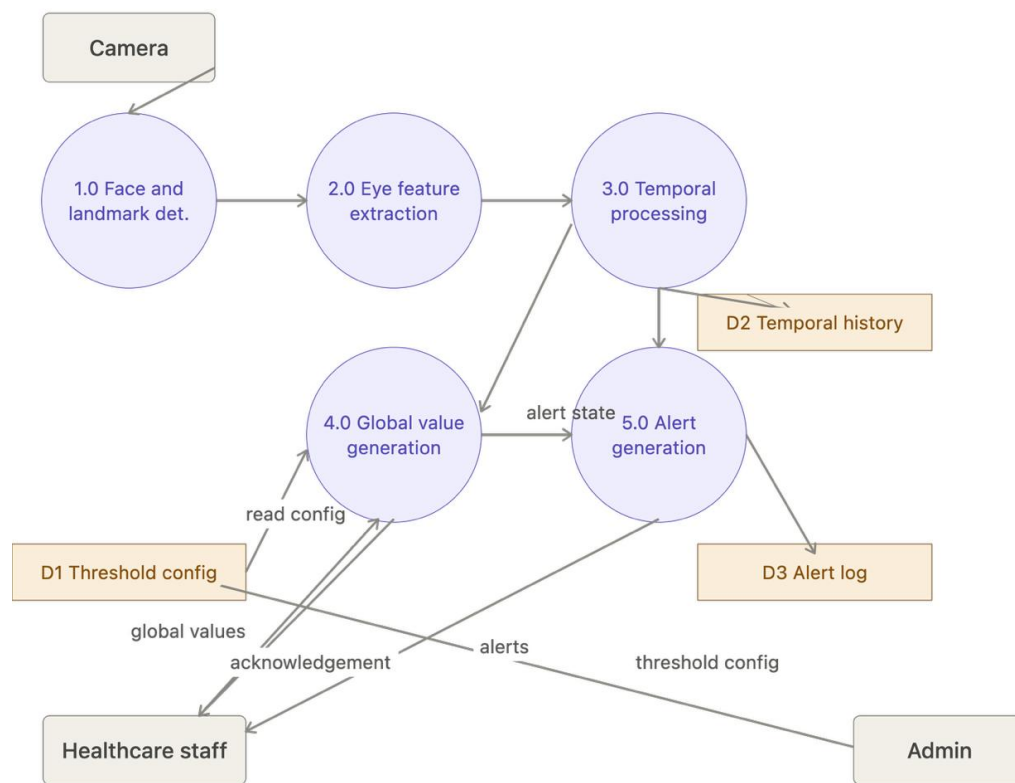


Figure 3.2: Level-1 Data Flow Diagram of the Face + Eye processing pipeline.

The Level-1 DFD decomposes the system into face and landmark processing, eye-feature extraction, temporal processing, global-value generation, and alert generation.

3.4 Use Case Design

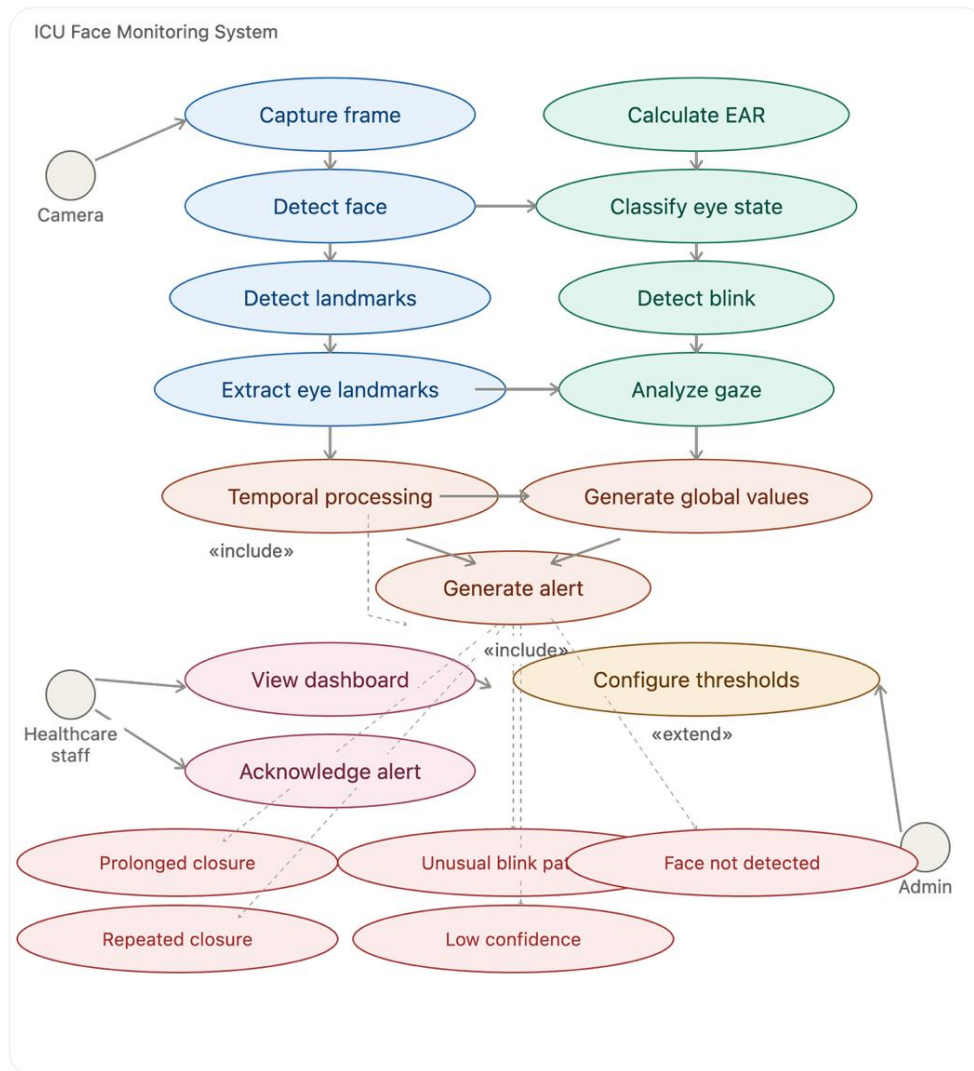


Figure 3.3: Use Case Diagram of the ICU Face Monitoring System.

The use-case model identifies the Camera, Healthcare Staff, and Admin as the principal actors.

3.5 UML Class Design

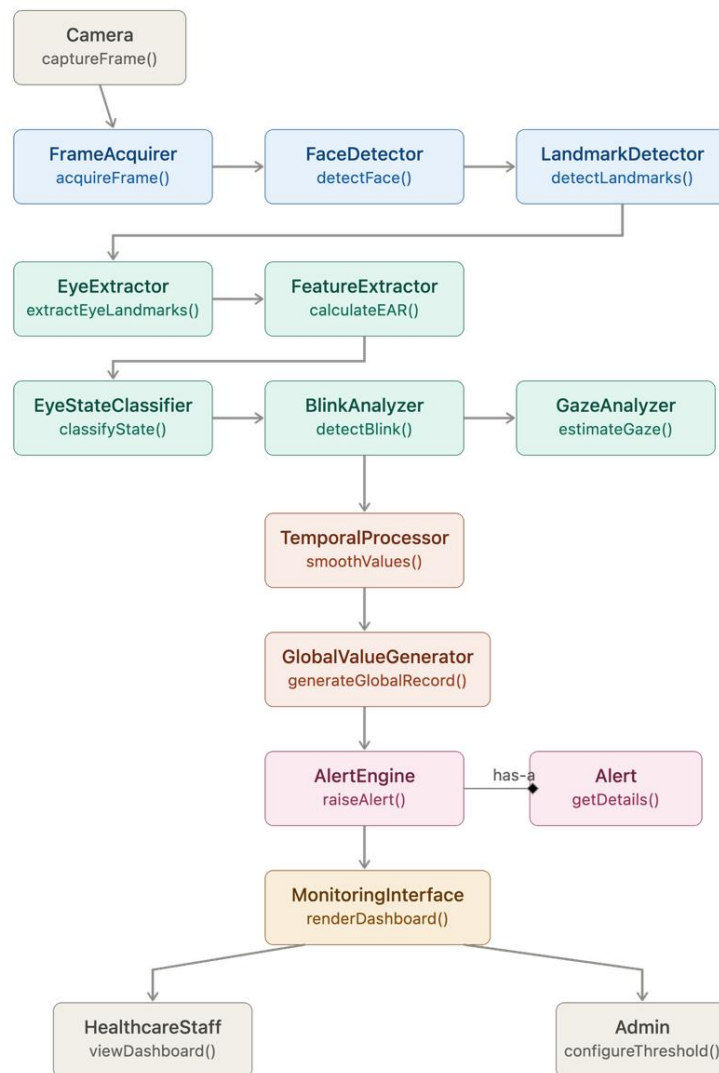


Figure 3.4: UML Class Diagram of the Face + Eye processing pipeline.

The class-level design separates frame acquisition, face detection, landmark extraction, eye processing, feature extraction, temporal analysis, global-value generation, and alert handling.

3.6 Activity Flow

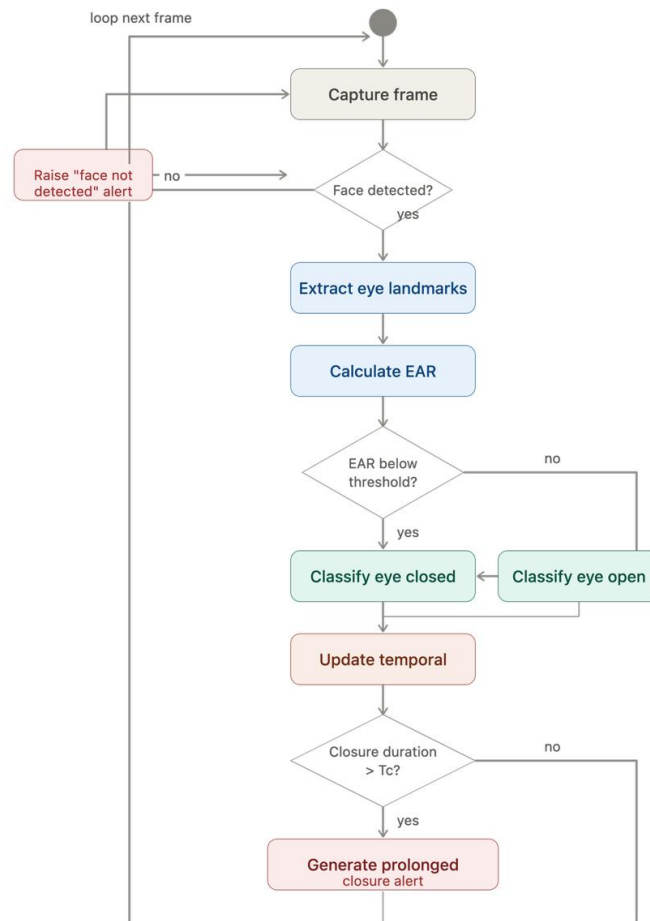


Figure 3.5: Activity diagram for per-frame processing and prolonged eye-closure alert generation.

The activity flow shows that each frame is checked for a usable face before eye features are calculated. Temporal information is updated before alert generation.

3.7 Sequence Flow

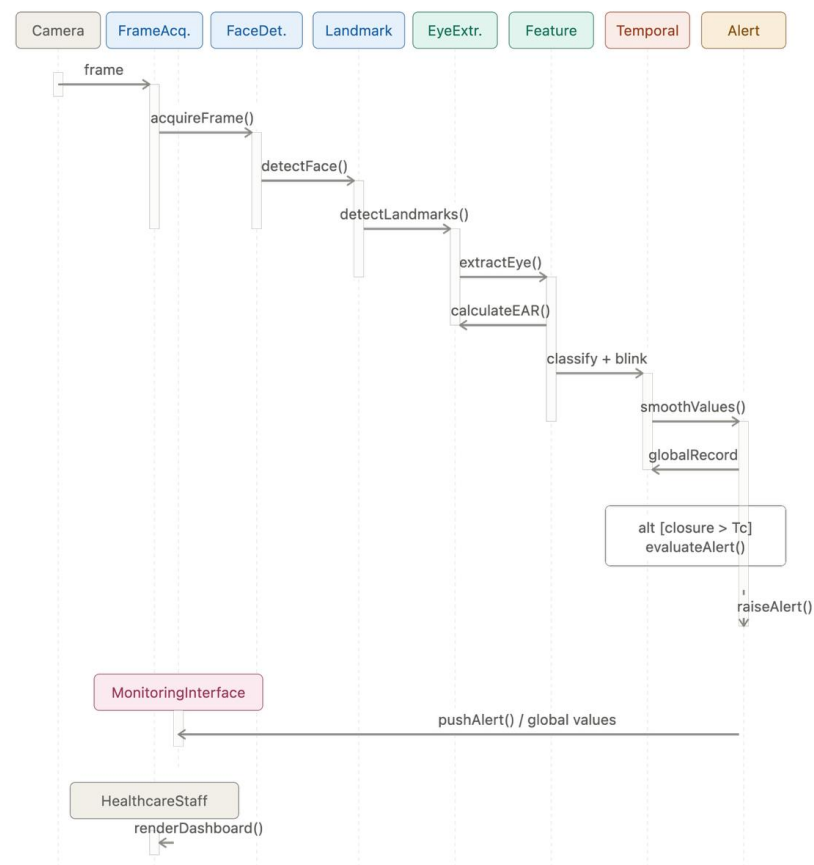


Figure 3.6: Sequence diagram showing the journey of a frame through the processing pipeline.

The sequence illustrates the movement of information from frame acquisition through feature calculation, temporal processing, alert evaluation, and monitoring delivery.

3.8 Technical Stack

Table 3.1: Technology stack

Technology	Role
Python	Primary implementation language.
OpenCV	Camera access and image processing.
MediaPipe Face Mesh	Facial landmark estimation.
NumPy	Numerical and geometric computation.
FastAPI	Backend API framework.
Uvicorn	ASGI application server.
Requests	HTTP communication from the Face + Eye pipeline.
Git	Version control and collaboration.

OpenCV is used for video acquisition and image processing, while MediaPipe provides facial geometry for downstream eye analysis [KAGG19; Ope26; Goo26].

Implementation

4.1 Project Structure

The implementation is divided into separate functional modules.

Listing 4.1: Representative project structure

```
1
2 Face + Eye Module/
3 |
4 +-- main.py
5 +-- backend_sender.py
6 |
7 +-- camera/
8 |   +-- camera_stream.py
9 |
10 +-- landmarks/
11 |   +-- landmark_extractor.py
12 |   +-- landmark_groups.py
13 |
14 +-- eye/
15 |   +-- gaze_estimator.py
16 |   +-- gaze_tracker.py
17 |
18 +-- l2cs/
19 |   +-- model.py
20 |   +-- pipeline.py
21 |   +-- results.py
22 |
23 +-- temporal/
24 |   +-- feature_buffer.py
25 |
26 +-- episodes/
```

```
27 |   +-- episode_detector.py
28 |
29 |   +-- confidence/
30 |     +-- confidence_estimator.py
31 |
32 |   +-- clinical_metrics/
33 |     +-- indices.py
34 |
35 |   +-- visualization/
36 |     +-- overlay.py
37 |
38 |   +-- logging_system/
39 |     +-- distress_logger.py
40 |
41 |   +-- backend/
42 |     +-- main.py
```

4.2 Face Detection

The first stage determines whether a usable face is present in the incoming frame. Subsequent processing is performed only when a valid face observation is available.

Face detection answers the question:

Where is the patient's face in the current frame?

Traditional real-time face detection approaches such as Viola–Jones demonstrated the feasibility of efficient face detection [VJ04].

If no usable face is detected, the frame is treated as a missing or invalid observation rather than being forced through the eye-analysis pipeline.

4.3 Facial Landmark Detection

After a face has been identified, facial landmarks are extracted. Each landmark represents a geometric location on the face. Together, these landmarks form a structured representation of facial geometry.

The principal purpose of this stage is to provide coordinates required by downstream eye and facial analysis.

4.4 Eye Landmark Extraction

The left and right eye regions are extracted from the facial landmark geometry. Both eyes are considered because one eye may occasionally have reduced visibility because of head orientation, illumination, or temporary occlusion.

The extracted eye landmarks provide the coordinates required for geometric measurements.

4.5 Eye Aspect Ratio

The principal geometric feature used by the eye-analysis pipeline is the Eye Aspect Ratio (EAR).

For six representative eye landmarks $p_1, p_2, p_3, p_4, p_5, p_6$:

$$EAR = \frac{\|p_2 - p_6\| + \|p_3 - p_5\|}{2\|p_1 - p_4\|} \quad (4.1)$$

where $\|\cdot\|$ represents Euclidean distance.

For an open eye, the vertical distances tend to be larger, producing a higher EAR. As the eye closes, the vertical distances decrease and EAR tends to become smaller.

EAR-based blink detection using facial landmarks is a recognised computer-vision technique [SČ16].

For the two eyes, the average EAR can be represented as:

$$EAR_{\text{avg}} = \frac{EAR_L + EAR_R}{2} \quad (4.2)$$

4.6 Eye-State Classification

The calculated EAR can be mapped to an observable eye state using a configured threshold.

Table 4.1: EAR-based eye-state interpretation

EAR Condition	Eye State
EAR above configured threshold	Open
EAR near configured threshold	Partially closed
EAR below configured threshold	Closed

The threshold is an implementation parameter rather than a universal medical constant.

4.7 Blink Detection

A blink is treated as a temporal transition:

$$\text{Open} \rightarrow \text{Closing} \rightarrow \text{Closed} \rightarrow \text{Opening} \rightarrow \text{Open}$$

Rather than declaring a blink from a single frame, the system observes eye measurements across time.

4.8 Temporal Processing

Facial and ocular measurements are treated as time-dependent observations.

For a sequence:

$$x_1, x_2, \dots, x_n$$

a moving average over the latest N measurements is:

$$\bar{x}_t = \frac{1}{N} \sum_{i=0}^{N-1} x_{t-i} \quad (4.3)$$

Temporal smoothing reduces the effect of isolated noisy observations.

4.9 Gaze Estimation

The system also incorporates gaze-related information.

Gaze is represented as relative behaviour rather than as an exact clinical gaze measurement.

Possible states include:

- left;
- right;
- upward;
- downward;
- centre; and
- uncertain.

4.10 Confidence Estimation

Confidence provides an indication of the reliability of the current observation.

Relevant factors include:

- face-detection quality;
- landmark quality;
- eye visibility;
- head pose;
- illumination; and
- temporary occlusion.

4.11 Global Monitoring State

The low-level outputs are consolidated into a structured monitoring record.

A representative structure is:

```
1  {  
2      "eye_state": "open",  
3      "left_ear": 0.31,  
4      "right_ear": 0.30,  
5      "average_ear": 0.305,  
6      "blink_count": 2,  
7      "closure_duration": 0.0,  
8      "gaze_state": "center",  
9      "confidence": 0.94,  
10     "alert_state": "normal"  
11 }
```

4.12 Alert Generation

The alert layer converts temporally processed information into observation-level monitoring events.

For prolonged eye closure:

$$Alert = \begin{cases} 1, & \text{if closure duration} > T_c, \\ 0, & \text{otherwise.} \end{cases} \quad (4.4)$$

where T_c represents the configured closure-duration threshold.

Table 4.2: Observation-level alert categories

Alert	Description
Prolonged Eye Closure	Eye closure persists beyond the configured duration.
Repeated Closure	Multiple closure events occur in a recent period.
Unusual Blink Pattern	Blink behaviour differs from configured criteria.
Low Confidence	Face/eye observations are considered unreliable.
Face Not Detected	A usable face cannot currently be identified.

Backend Integration

5.1 Integration Objective

The backend provides a communication boundary between the Face + Eye processing engine and downstream monitoring applications.

The architecture separates:

1. **Processing Layer:** Video handling, face detection, landmarks, eye features, temporal processing, gaze, confidence, global values, and alerts.
2. **Communication Layer:** HTTP requests, JSON payloads, frame uploads, endpoint handling, and response generation.

5.2 FastAPI Backend

The backend is implemented using FastAPI and is served using Uvicorn.

The local development server is started using:

```
1 uvicorn backend.main:app --host 0.0.0.0 --port 8000 --reload
```

The service is accessible locally through:

```
1 http://127.0.0.1:8000
```

The interactive API documentation is available through:

```
1 http://127.0.0.1:8000/docs
```

5.3 Feature Communication

The Face + Eye processing pipeline sends structured monitoring information to:

```
1 POST /api/face-eye/features
```

The request body contains the processed feature information in JSON format.

5.4 Dashboard Frame Communication

Dashboard frames are transmitted separately through:

```
1 POST /api/stream/face
```

Frames are encoded as JPEG before transmission.

5.5 Asynchronous Frame Delivery

Dashboard frame transmission is separated from the primary processing path.

The sender maintains only the latest frame. A background thread waits for new frames and uploads them to the backend.

This prevents old frames from accumulating in an unbounded queue and reduces the effect of streaming delays on the main processing loop.

5.6 Backend API

The backend receives Face + Eye feature information and dashboard frames.

A simplified implementation is:

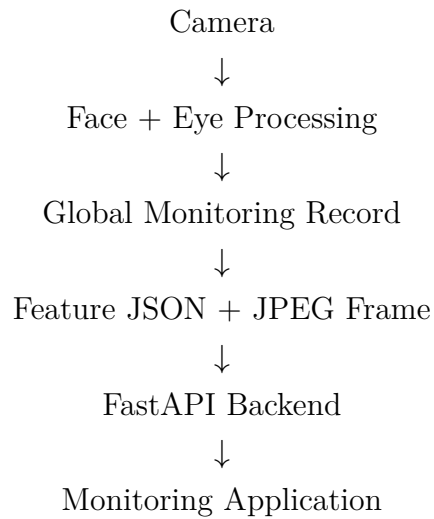
```
1 from fastapi import FastAPI, UploadFile, File
2 from fastapi.responses import StreamingResponse
3 import io
4
5 app = FastAPI()
6
7 face_eye_data = {}
8 posture_data = {}
9 latest_frame = None
10
11
12 @app.post("/api/face-eye/features")
13 def receive_face_eye(data: dict):
14
15     global face_eye_data
```

```
16
17     face_eye_data = data
18
19     return {"received": True}
20
21
22 @app.get("/api/face-eye/features")
23 def get_face_eye():
24
25     return face_eye_data
26
27
28 @app.post("/posture")
29 def receive_posture(data: dict):
30
31     global posture_data
32
33     posture_data = data
34
35     return {"received": True}
36
37
38 @app.get("/posture")
39 def get_posture():
40
41     return posture_data
42
43
44 @app.get("/fusion")
45 def fusion():
46
47     return {
48         "face_eye": face_eye_data,
49         "posture": posture_data
50     }
51
```

```
52
53 @app.post("/api/stream/face")
54 async def receive_frame(
55     frame: UploadFile = File(...)
56 ):
57
58     global latest_frame
59
60     latest_frame = await frame.read()
61
62     return {"received": True}
63
64
65 @app.get("/api/stream/face")
66 def get_frame():
67
68     if latest_frame is None:
69         return {"error": "No frame available"}
70
71     return StreamingResponse(
72         io.BytesIO(latest_frame),
73         media_type="image/jpeg"
74     )
```

5.7 End-to-End Integration Flow

The completed communication architecture is:



5.8 Failure Handling

Backend failures should not terminate the primary computer-vision pipeline.

The feature sender catches HTTP request exceptions, while the frame sender operates in a separate daemon thread.

This provides separation between computer-vision processing, clinical feature communication, and dashboard frame streaming.

Testing and Evaluation

6.1 Testing Methodology

Testing is organised at multiple levels.

Table 6.1: Testing methodology

Test Area	Representative Tests
Face Detection	Normal illumination, low illumination, different distances, face orientation, temporary occlusion.
Eye Landmarks	Open eyes, closed eyes, blinking, partial visibility, head movement.
EAR Features	Open versus closed eyes and left/right eye consistency.
Temporal Processing	Short blink versus sustained closure and noisy frames.
Alert Generation	Correct triggering, alert clearing, and avoidance of excessive alerts.
Backend	Server startup, endpoint accessibility, JSON validation, invalid requests, and communication failures.
Streaming	JPEG encoding, frame upload, latest-frame replacement, and streaming failure isolation.
End-to-End	Camera frame through processing, global state, alert generation, and backend delivery.

6.2 Performance Metrics

The principal performance metrics are FPS, precision, recall, F1, latency, and detection accuracy.

$$FPS = \frac{\text{Number of processed frames}}{\text{Processing time}} \quad (6.1)$$

$$Precision = \frac{TP}{TP + FP} \quad (6.2)$$

$$Recall = \frac{TP}{TP + FN} \quad (6.3)$$

$$F1 = 2 \frac{Precision \times Recall}{Precision + Recall} \quad (6.4)$$

Latency represents the time between frame acquisition and the availability of the corresponding processed information.

6.3 Evaluation Targets

Table 6.2: Proposed evaluation targets

Metric	Target
Processing latency	Below approximately 50 ms per processing step.
Face detection accuracy	Above 95% on selected test data.
Blink detection accuracy	Above 90% on labelled sequences.
Throughput	Approximately 15 FPS or higher.

These values are evaluation targets and must not be presented as measured results unless experimentally validated.

6.4 Backend Integration Validation

The backend integration can be validated through the following sequence:

1. Start the FastAPI application using Uvicorn.
2. Open <http://127.0.0.1:8000/docs>.

3. Confirm that the Face + Eye endpoints are registered.
4. Start the Face + Eye pipeline.
5. Confirm that feature POST requests reach the backend.
6. Confirm that frame POST requests reach the streaming endpoint.
7. Query the fusion endpoint.
8. Verify that Face + Eye information is present in the returned data.

6.5 Current Results

The implemented work establishes the processing and communication pipeline from camera input through the backend boundary.

The transformation achieved is:

Camera Frame $\rightarrow$ Face Information $\rightarrow$ Landmarks $\rightarrow$ Eye Measurements $\rightarrow$ Temporal Information $\rightarrow$ Global Monitoring State $\rightarrow$ Alert Information $\rightarrow$ Backend Data

A complete numerical accuracy evaluation requires labelled test data, defined ground truth, repeatable conditions, and systematic measurement.

Discussion

7.1 Engineering Significance

The principal engineering contribution is the transformation of raw visual input into a structured monitoring representation.

The system creates an abstraction boundary around the processed monitoring record, improving maintainability, testability, integration, modularity, and future extensibility.

7.2 Importance of Temporal Reasoning

Temporal reasoning is a central design decision.

A single low EAR measurement may result from a normal blink, landmark noise, head movement, poor illumination, temporary occlusion, or sustained eye closure.

Using recent history and duration allows the system to distinguish isolated measurements from persistent conditions.

7.3 Processing and Communication Separation

The processing engine and backend are deliberately separated.

The processing layer is responsible for video acquisition, face detection, landmark extraction, eye features, temporal analysis, gaze, confidence, global values, and alerts.

The backend is responsible for receiving structured data, serving processed data, receiving frames, and exposing information to downstream clients.

7.4 Asynchronous Frame Streaming

Dashboard frame transmission is deliberately separated from the clinical feature path.

Only the newest frame is retained for transmission. This prevents a slow network connection from producing an unbounded backlog of old frames.

7.5 Limitations

The current system has several limitations:

- sensitivity to poor illumination;
- sensitivity to large head-pose changes;
- possible occlusion from hands or medical equipment;
- dependency on camera placement;
- dependency on landmark quality;
- fixed thresholds rather than personalised baselines;
- relative rather than clinically calibrated gaze estimation;
- absence of large-scale clinical validation; and
- dependency on backend/network availability.

The output should therefore be considered supplementary monitoring information rather than an independent medical diagnosis.

Conclusion and Future Work

8.1 Summary

The Face + Eye component of the ICU Patient Monitoring System implements a modular computer-vision pipeline for non-contact observation of facial and ocular behaviour.

The processing chain begins with camera input and progresses through face detection, facial landmarks, eye landmarks, EAR-based feature extraction, eye-state analysis, blink analysis, temporal processing, gaze estimation, confidence estimation, global-value generation, alert generation, and backend integration.

8.2 Key Findings

1. **Temporal processing improves robustness.**
2. **Modularity improves maintainability.**
3. **Structured API communication simplifies integration.**
4. **Asynchronous streaming protects the main pipeline.**
5. **The system is supplementary and requires validation before clinical deployment.**

8.3 Future Work

1. Patient-specific calibration.
2. Improved temporal modelling.
3. Improved gaze estimation.
4. Multi-camera deployment.

5. Historical monitoring.
6. Scalable backend architecture.
7. Advanced alert prioritisation.
8. Systematic evaluation using labelled sequences.
9. Production dashboard integration.
10. Security and privacy controls.

8.4 Final Remarks

The project demonstrates that a modular computer-vision system can transform camera observations into structured, time-aware monitoring information and expose that information through a backend service.

The strength of the implementation lies in the integration of multiple processing stages rather than any single feature.

The system should be considered a supplementary monitoring and decision-support component. Appropriate technical, clinical, privacy, and security validation is required before deployment in a real ICU environment.

Bibliography

- [BDMS16] Vitoantonio Bevilacqua, Donato D’Ambruoso, Giacomo Mandolino, and Mariapia Suma. “Vision-Based Patient Monitoring: A Comprehensive Review of Algorithms and Technologies”. In: *Journal of Ambient Intelligence and Humanized Computing* (2016) (cit. on p. 1).
- [DMS+19] Anis Davoudi, Kumar Rohit Malhotra, Benjamin Shickel, et al. “Intelligent ICU for Autonomous Patient Monitoring Using Pervasive Sensing and Deep Learning”. In: *Scientific Reports* 9.1 (2019), p. 8020 (cit. on p. 1).
- [Goo26] Google MediaPipe. *MediaPipe Face Mesh*. 2026. URL: <https://github.com/google-ai-edge/mediapipe> (cit. on p. 16).
- [KAGG19] Yury Kartynnik, Artsiom Ablavatski, Ivan Grishchenko, and Matthias Grundmann. “Real-Time Facial Surface Geometry from Monocular Video on Mobile GPUs”. In: *arXiv preprint arXiv:1907.06724* (2019) (cit. on p. 16).
- [Ope26] OpenCV. *Open Source Computer Vision Library*. 2026. URL: <https://opencv.org> (cit. on p. 16).
- [SČ16] Tereza Soukupová and Jan Čech. “Real-Time Eye Blink Detection Using Facial Landmarks”. In: *21st Computer Vision Winter Workshop*. Rimske Toplice, Slovenia, 2016 (cit. on p. 19).
- [VJ04] Paul Viola and Michael J. Jones. “Robust Real-Time Face Detection”. In: *International Journal of Computer Vision* 57.2 (2004), pp. 137–154 (cit. on p. 18).